How to Model Cryptoeconomic Adversaries in DeFi

Abhimanyu Nag

MITACS Fellow, University of Alberta
DeFi Security Summit 2026

November 2026

DeFi auctions allocate scarce opportunities

- Liquidation auctions sell collateral to repay debt.
- Trading auctions select how signed orders will execute.
- Blockspace auctions determine which transactions and blocks get included.

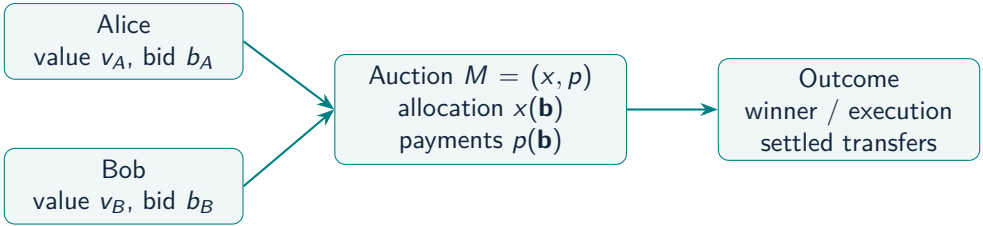
An auction specifies **who can bid**, **who receives what**, and **who pays**.

Main point

Emphasis on Sybils: how can one economic actor manipulate an auction through several identities?

Examples: Maker collateral liquidation, CoW Protocol batch auctions, Flashbots MEV-Boost.

An auction has an allocation rule and a payment rule



Single-item example: second-price auction

Bids $(10, 7, 4)$: the highest bidder wins and pays 7.
Expected utility: $u_i = v_i x_i(\mathbf{b}) - p_i(\mathbf{b})$.

Here x_i is a winning probability and p_i is an expected payment.
Onchain implementations also specify block reward, ordering, deadlines, deposits, and settlement.

Different DeFi auctions expose different choices

Example	Auction design	Adversarial question
Sky / Maker Liquidation 2.0	Dutch auction: collateral price declines with time, purchases settle instantly.	Who can buy first? What happens when prices reset or oracle inputs lag?
CoW Protocol	Orders collected offchain. Solvers compete to propose valid batch settlements.	Can a solver manipulate the competition or exploit settlement constraints?
Flashbots MEV-Boost	Builders offer blocks and payments through relays. Proposers select among delivered bids.	Who controls order flow, bidding timing, and block delivery?

MEV mitigation does not imply resistance to every economic attack. Some vulnerabilities are design specific.

An attack can exploit code or the economic rules

Code attack

- Reentrancy or incorrect access control.
- Accounting and settlement inconsistencies.
- Implementation violates its specification.

Economic attack

- False identities and strategic bids.
- Collusion, bribery, or ordering control.
- Rules execute correctly but reward manipulation.

Model the attacker: objective, information, available actions, resources, and costs.

$$a^* \in \arg \max_{a \in \mathcal{A}(I,R)} \mathbb{E}[U(a) \mid I] - \mathrm{Cost}(a).$$

I: information available before acting. R: capital, identities, compute, and ordering access.

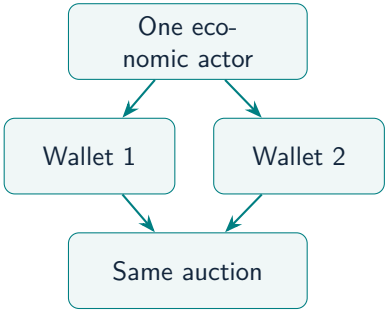
One actor can submit several apparently independent bids

Truthfulness: one identity cannot gain by misreporting its value. Contention here

Sybil-proofness: one actor cannot gain by replacing its bid with several bids.

For a single item, the actor values winning once and pays for **all** its identities:

$$U_{\text{split}} = v_i \sum_{\ell=1}^{m} x_\ell - \sum_{\ell=1}^{m} p_\ell.$$



Main point

An auction which is safe for one bid $\neq$ Safe for multiple bids

Literature: Yokoo, Pseudonymous Bidding in Combinatorial Auctions, Pan et al. On Sybil Proof Mechanisms

A complete Sybil attack on a toy auction

Rule: one prize worth 120, uniformly allocated among registered identities as a lottery.

Baseline: three identities

Attacker + two honest participants.

$\Pr(\text{attacker wins}) = 1/3$.
Expected utility: $120/3 = \$40$.

Deviation: four identities

Attacker registers two identities.

$\Pr(\text{attacker wins}) = 2/4$.
Expected utility: $120/2 = \$60$.

End to end on a local test chain: deploy lottery, fund test wallets, register the two attacker wallets, settle with unbiased randomness, aggregate their outcomes.

Main point

Gross gain: \$20. The attack is profitable when incremental fees and identity costs are below \$20.

Attack search is an optimization problem

Hold the mechanism fixed. For one attacker, search over **two freely chosen replacement bids**:

$$\Delta_i(s,t) = v_i\big[x_1(s,t,\mathbf{b}_{-i}) + x_2(s,t,\mathbf{b}_{-i})\big] - p_1(s,t,\mathbf{b}_{-i}) - p_2(s,t,\mathbf{b}_{-i}) - U_i^{\mathrm{truthful}}.$$

- Search grids, ties, reserve boundaries, and several local restarts.
- Use gradients for smooth mechanisms. Use derivative-free search for discrete settlement.
- Recompute candidate gains with accurate payments and replay the actual implementation.
- Add gas, deposits, timing, and funding constraints to obtain net gain.

Main point

Second Price Auction is the only auction to be proven Sybil Proof for now.

Pan et al. On Sybil Proof Mechanisms

AI agents as economic attackers

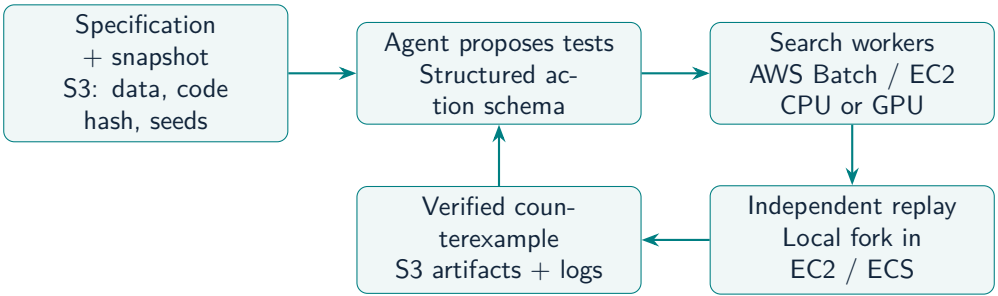
Reasoning: hypothesis and evidence

1. **Read the rules:** identify allocation, payments, and entry conditions.
2. **Form a hypothesis:** "An extra identity may increase my combined winning probability."
3. **Evaluate the deviation:** compare expected utility against one-identity participation, including all costs.
4. **Revise from feedback:** change bids or identity count when simulation rejects the hypothesis.

Implementation: an agent with tools

- **Inputs:** contract code, auction state, attacker capital, and available observations.
- **Planner:** an LLM proposes a structured action: identities, bids, and submission times.
- **Tools:** Python optimizer searches parameters; a local EVM fork executes transactions.
- **Verifier:** aggregates wallet outcomes, checks feasibility, and records net gain.

An agent can coordinate search and reproducible replay



Step Functions orchestrates jobs. CloudWatch records execution. IAM scopes worker access.
Output: reproducible bids, wallet mapping, baseline, net gain, transaction trace.
agents can help write and run tests. Reliable discovery still depends on the model, optimizer, and independent verification.

Learning a mechanism against an attacker

Inner problem: freeze mechanism parameters θ, search for profitable deviations.

$$\widehat{r}_i(\theta) = \max\{0, \max_{(s,t)\in\mathcal{S}} \Delta_i^{\theta}(s,t)\}.$$

Outer problem: improve revenue while penalizing the observed regret.

$$\min_{\theta} -\widehat{R}(\theta) + \lambda(\widehat{r}(\theta) - \epsilon) + \frac{\rho}{2}\big[\max\{0, \widehat{r}(\theta) - \epsilon\}\big]^2.$$

Final Thoughts

1. **Specify ownership.** Several addresses may share one objective.
2. **Specify the feasible deviation.** Include what the attacker knows and can afford.
3. **Compare utilities.** Aggregate allocations, payments, and execution costs.
4. **Separate evidence from guarantees.** Search finds counterexamples. Proof establishes resistance within a stated model.

Main point

Economic security needs an explicit adversarial model

Thank you

Questions? Come find me. Looking to collab!

Abhimanyu Nag
abhi.nag1601@gmail.com